Ch 01: How a Program Works

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026

EAST TEXAS A&M

How a Program Works

Concept

A computer's CPU can only understand instructions that are written in machine language. Because people find it very difficult to write entire programs in machine language, other programming languages have been invented.

EAST TEXAS A&M

How a Program Works (1 of 2)

- CPU designed to perform simple operations on pieces of data
  - reading data, adding, subtracting, multiplying and dividing numbers
  - Understands instructions written in machine language (instruction set)
    - Each brand of CPU has its own instruction set
  - Instructions are simple, to carry out meaningful calculation, CPU must perform many operations
- Program must be copied from secondary memory to RAM each time CPU executes it
- CPU executes program in cycle
  - **Fetch**: read the next instruction from RAM into CPU
  - **Decode**: CPU decodes fetched instruction to determine operation to perform
  - **Execute**: perform the operation



How a Program Works (2 of 2)

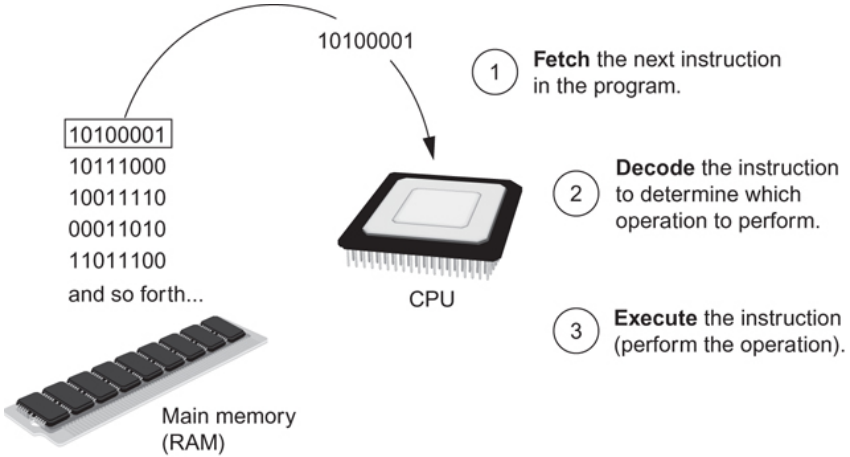


Figure 1: The fetch-decode-execute cycle (Gaddis, 2021, pg.37)

EAST TEXAS A&M

From Machine Language to Assembly Language

- Impractical for people to write in machine language
- **Assembly language**: uses words (mnemonics) for instructions instead of binary numbers
  - Easier for programmers to work with
- **Assembler**: translates assembly language to machine language for execution by CPU

x86 Assembly Example

```
section .data
    msg db 'Hello, World!', 0xA  ; Message string with newline
    len equ $ - msg              ; Length of the string
section .text
    global _start
_start:
    mov eax, 4          ; System call number for sys_write
    mov ebx, 1          ; File descriptor (stdout)
    mov ecx, msg        ; Pointer to the message
    mov edx, len        ; Length of the message
    int 0x80            ; Call kernel
    mov eax, 1          ; System call number for sys_exit
    xor ebx, ebx        ; Exit code 0
    int 0x80            ; Call kernel
```

EAST TEXAS A&M

High-Level Languages

- **Low-level language**: close in nature to machine language
  - Example: assembly language
- **Medium-level system language**: a "high-level" language, but still relatively directly translatable to assembly for different architectures
- **High-level language**: allows simple create of powerful and complex programs
  - No need to know how CPU works or write large number of instructions
  - More intuitive to understand

C language Example

```c
#include <stdio.h>
int main() {
    int n, i;
    unsigned long long fact = 1;
    printf("Enter an integer: ");
    scanf("%d", &n);

    // shows error if the user enters a negative integer
    if (n < 0)
        printf("Error! Factorial of a negative number doesn't exist.");
    else {
        for (i = 1; i <= n; ++i) {
            fact *= i;
        }
```

Keywords, Operators and Syntax: an Overview

- **Keywords**: predefined words used to write program in high-level language
  - Each keyword has specific meaning
- **Operators**: perform operations on data
  - Example: math operators to perform arithmetic
- **Syntax**: set of rules to be followed when writing program
- **Statement**: individual instruction used in high-level language

The Python keywords

and	continue	finally	is	raise
as	def	for	lambda	return
assert	del	from	None	True
async	elif	global	nonlocal	try
await	else	if	not	while
break	except	import	or	with
class	False	in	pass	yield

EAST TEXAS A&M

Compilers and Interpreters (1 of 3)

- Programs written in high-level languages must be translated into machine language to be executed
- **Compiler**: translates high-level language program into separate machine language program
  - Machine language program can be executed at any time
- **Interpreter**: translates and executes instructions in high-level language programs
  - Used by Python language
  - Interprets one instruction at a time
  - No separate machine language program
- **Source code**: statements written by a programmer
  - **Syntax error**: prevents code from being translated



EAST TEXAS A&M

Compilers and Interpreters (2 of 3)

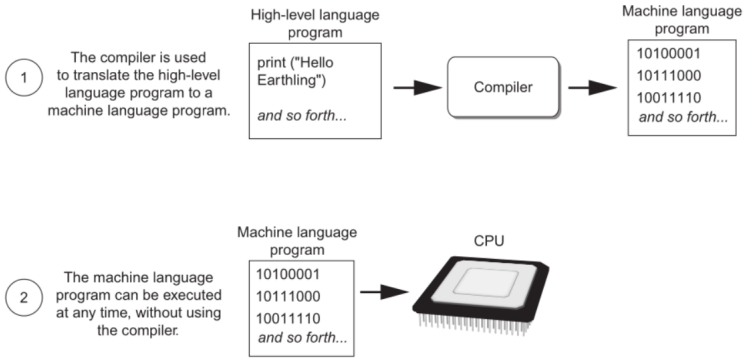


Figure 2: Compiling a high-level program (Gaddis, 2021, pg.40)

EAST TEXAS A&M

Compilers and Interpreters (3 of 3)

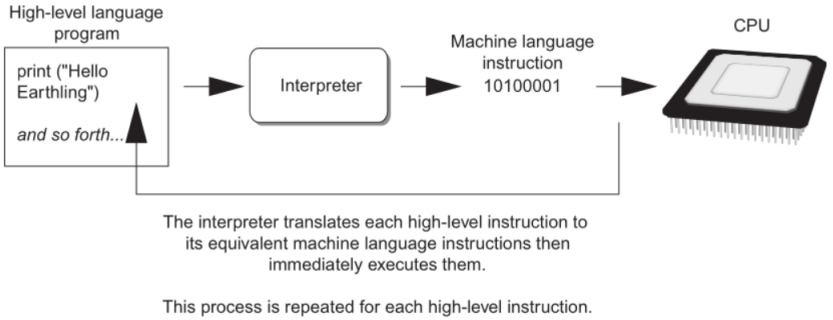


Figure 3: Executing a high-level program with an interpreter (Gaddis, 2021, pg.41)

EAST TEXAS A&M

Summary

- A CPU only understands machine instructions (sequence of binary numbers)
- The CPU runs machine instructions by fetching, decoding and executing one at a time
- Machine language is impractical for people to write directly, can use Assembly
- Most applications will be written in a high-level language, for speed and simplicity
- A high-level language is made of keywords and operators, with a defined Syntax in order to create statements in the high-level language
- A high-level language has to be translated to machine language
  - Either it is compiled once to a machine language executable
  - Or an interpreter is used to interpret and translate statements



EAST TEXAS A&M

References

Gaddis, T. (2021). *Starting out with Python* (5th). Pearson.

EAST TEXAS A&M